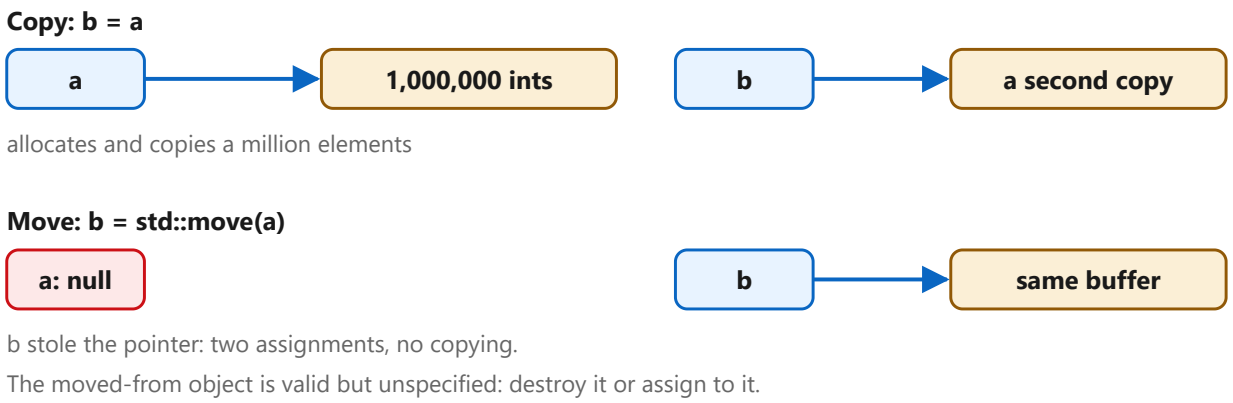
Copy versus Move, and std::move

What copying costs, what `std::move` really does (a cast, not a move), what a moved-from object is, and when a move silently becomes a copy. Assumes Lvalues and Rvalues.

What a copy costs

Copying a vector of a million ints allocates a new buffer and copies a million values. If the source is a temporary that is about to die anyway, that work is wasted. A **move** instead *steals the inside*: it copies the pointer and size, and leaves the source empty.



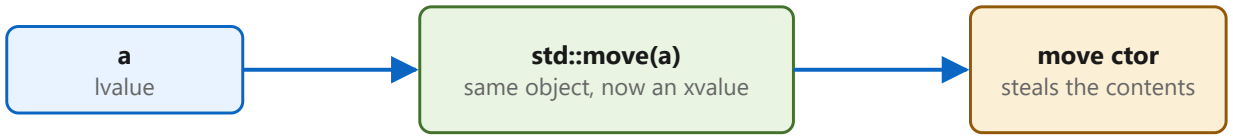
Copy: two buffers. Move: one buffer, a new owner.

	COPY	MOVE
allocates memory	yes	no
copies elements	all of them, O(n)	none, O(1)
source afterwards	unchanged	valid but unspecified (usually empty)
used when the source is	an lvalue you still need	an rvalue, or `std::move(x)`

Not every type benefits. A move of an `int` or a small struct is just a copy; the gain is for types that **own a resource**: vectors, strings, `unique_ptr`, file handles.

What `std::move` really does

The name is misleading: `std::move(x)` **moves nothing**. It is a cast, equal to `static_cast<T&&>(x)`, that changes the expression from an lvalue to an rvalue. The *move constructor or move assignment* chosen afterwards does the stealing.



No data moves in the middle box. Overload resolution then picks the && overload.

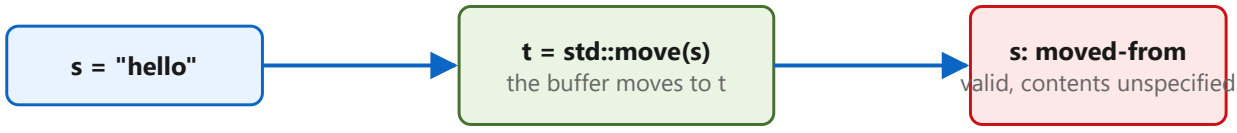
std::move is a cast, nothing more.

```
std::vector<int> a(1000, 7);
// lvalue: copy constructor, 1000 ints copied
std::vector<int> b = a;
// xvalue: move constructor, only pointers change
std::vector<int> c = std::move(a);
```

KEY POINT "What does `std::move` do?" Nothing at run time: an unconditional cast to an rvalue reference. Whether anything is stolen depends on the type having a move operation.

Moved-from objects

After `auto t = std::move(s);` the object `s` still exists and its destructor will still run. The standard promises only that it is in a **valid but unspecified** state.



Safe: destroy it, assign to it, call functions that need no precondition (size(), empty(), clear()).

A moved-from object is valid but unspecified.

```
std::string s = "hello";
std::string t = std::move(s);
// compiled: s.size() == 0 here for std::string, but do not rely on it
s = "new value";            // assigning always works: s is usable again
```

YOU MAY	YOU MAY NOT RELY ON
destroy it	its old value
assign a new value to it	its size or contents
call functions with no preconditions: `clear()`, `empty()`, `size()`	reading elements

- For standard types, a moved-from object is usually empty, but only *your own* types can promise more.
- Move only at the **last use**. After `auto b = std::move(a);` treat `a` as empty.
- Moving inside a loop is a bug: the second iteration sees a moved-from object.

When `std::move` copies anyway

Two situations make `std::move` quietly do nothing useful.

SITUATION	WHAT HAPPENS	WHY
`std::move` on a `const` object	**copies** (compiled: 1 copy, 0 moves)	a `const T&&` cannot bind to `T&&`, so the copy constructor is chosen
a type with no move operations	copies	the `const T&` overload accepts an rvalue as well
a type with a user-declared destructor or copy operation	copies	the compiler does not generate moves then (Rule of Five)

```
const std::string cs = "text";
// compiles, but COPIES: a const object cannot be stolen from
std::string x = std::move(cs);
```

SILENT None of these is an error. The code is correct and slower than you think. Never return `const` values by value either: it disables the move at the caller.

Which standard types are cheap to move

A move is only cheap when there is a **pointer to hand over**. Knowing which standard types have one helps you predict the cost.

vector, string (long), unique_ptr
move = copy a few pointers: O(1)

array, small string, plain struct
move = copy each element: O(n)

sizeof (compiled, 64-bit): vector 24 bytes, string 32 bytes, unique_ptr 8 bytes.

A std::array of 3 objects moved 3 times; a vector of 3 moved 0 elements (compiled).

Cheap moves need a pointer to hand over: the object must own heap memory.

TYPE	MOVE COST	WHY
`std::vector`, `std::map`, `std::list`	O(1)	the elements live on the heap: only the pointers change hands
`std::string`	O(1) for long text, a copy for short text	short strings are stored inside the object (small string optimization)
`std::unique_ptr`	O(1), and the only way to transfer it	copy is deleted; a move copies one pointer and nulls the source
`std::array<T, N>`	O(N)	the elements are stored inline, so each one is moved (compiled: 3 moves for N = 3)
`int`, `double`, plain structs	same as a copy	nothing to steal

```
std::vector<C> v(3);
// 0 element moves, 0 copies (compiled): the buffer just changes owner
auto v2 = std::move(v);
std::array<C, 3> a;
// 3 element moves (compiled): inline storage cannot be stolen
auto a2 = std::move(a);
```

Containers of movable elements use moves internally: `push_back` of a temporary, growth, `std::sort` swapping elements and `std::erase` shifting them. That is why moves matter even when you never write `std::move`.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
What a copy costs	C++ Move Semantics	1.1 Motivation for move semantics	21 to 36
What std::move really does	Effective Modern C++	Item 23, Understand std::move and std::forward	176 to 182
	C++ Move Semantics	1.2 to 1.4 Implementing move semantics; copying as a fallback; const objects	36 to 41
Moved-from objects	C++ Move Semantics	6.1 to 6.2 Moved-from states (required and guaranteed states; destructible and assignable)	107 to 115
When std::move copies anyway	C++ Move Semantics	1.2 to 1.4 Implementing move semantics; copying as a fallback; const objects	36 to 41